

JavaScript Reflect

[Back to JS page](#)

Table of Contents

- [JavaScript Reflect](#)
 - [Reflect.has\(\)](#)
 - [Example 1](#)
 - [Reflect.get\(\) and Reflect.set\(\)](#)
 - [Example 2](#)
 - [Reflect.deleteProperty\(\)](#)
 - [Example 3](#)
 - [Reflect.apply\(\)](#)
 - [Example 4](#)
 - [Reflect.construct\(\)](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

The Reflect Object

The `Reflect` object provides methods for interceptable JavaScript operations. It makes metaprogramming easier by providing a unified way to perform operations like getting, setting, and deleting properties.

Before `Reflect`, developers had to use different syntax for these operations. With `Reflect`, you get a consistent API.

Reflect.has()

`Reflect.has()` checks if a property exists in an object. It's similar to the `in` operator:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.has()</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Check if properties exist using Reflect.has()
let text = "Checking properties using Reflect.has():<br>";
text += "Has 'firstName': " + Reflect.has(person, "firstName") + "<br>";
text += "Has 'email': " + Reflect.has(person, "email") + "<br>";
text += "Has 'age': " + Reflect.has(person, "age");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Reflect.get() and Reflect.set()

`Reflect.get()` gets a property value. `Reflect.set()` sets a property value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.get() and Reflect.set(</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 30
};

// Get a property using Reflect.get()
let name = Reflect.get(person, "firstName");

// Set a property using Reflect.set()
Reflect.set(person, "age", 35);
Reflect.set(person, "email", "john@example.com");

document.getElementById("demo").innerHTML =
  "Reflect.get(person, 'firstName'): " + name + "<br>" +
  "Reflect.set(person, 'age', 35): " + Reflect.get(person, "age") + "<br>" +
  "Reflect.set(person, 'email', 'john@example.com'): " + Reflect.get(person, "email") + "<br>" +
  "All properties: " + Object.keys(person).join(", ");
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Reflect.deleteProperty()

`Reflect.deleteProperty()` deletes a property from an object. It's similar to the `delete` operator:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.deleteProperty()</h4>
<p id="demo"></p>

<script>
const car = {
  make: "Toyota",
  model: "Corolla",
  year: 2020,
  color: "blue"
};

let text = "Before delete:<br>";
text += Object.keys(car).join(", ") + "<br><br>";

// Delete a property using Reflect.deleteProperty()
Reflect.deleteProperty(car, "color");

text += "After deleteProperty(car, 'color'):<br>";
text += Object.keys(car).join(", ") + "<br><br>";

// Check if property still exists
text += "Has 'color': " + Reflect.has(car, "color");

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Reflect.apply()

`Reflect.apply()` calls a function with a given `this` value and arguments. It's similar to `Function.prototype.apply()` :

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.apply()</h4>
<p id="demo"></p>

<script>
function greet(greeting, punctuation) {
  return greeting + ", " + this.name + punctuation;
}

const person = { name: "John" };
const member = { name: "Jane" };

// Using Reflect.apply() with different 'this' values
let result1 = Reflect.apply(greet, person, ["Hello", "!!"]);
let result2 = Reflect.apply(greet, member, ["Hi", "!!"]);

document.getElementById("demo").innerHTML =
  "Reflect.apply(greet, person, ['Hello', '!!']): " + result1 + "<br>" +
  "Reflect.apply(greet, member, ['Hi', '!!']): " + result2;
</script>

</body>
</html>

```

Example 4

Result [View Example](#)

Reflect.construct()

`Reflect.construct()` creates a new instance of a constructor function. It's similar to the `new` keyword:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Reflect</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Reflect.construct()</h4>
<p id="demo"></p>

<script>
class Car {
  constructor(make, model, year) {
    this.make = make;
    this.model = model;
    this.year = year;
  }
  getInfo() {
    return this.year + " " + this.make + " " + this.model;
  }
}

// Using Reflect.construct() instead of 'new Car()'
const myCar = Reflect.construct(Car, ["Toyota", "Corolla", 2020]);
const yourCar = Reflect.construct(Car, ["Honda", "Civic", 2022]);

document.getElementById("demo").innerHTML =
  "Reflect.construct(Car, [...]) :<br>" +
  "My car: " + myCar.getInfo() + "<br>" +
  "Your car: " + yourCar.getInfo() + "<br><br>" +
  "Instanceof check: " + (myCar instanceof Car);
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Reflect](#)